Keycloak Workflow Diagrams

Deep beginner-friendly explanation of authentication, tokens, APIs, authorization, and production flow

Important scope note

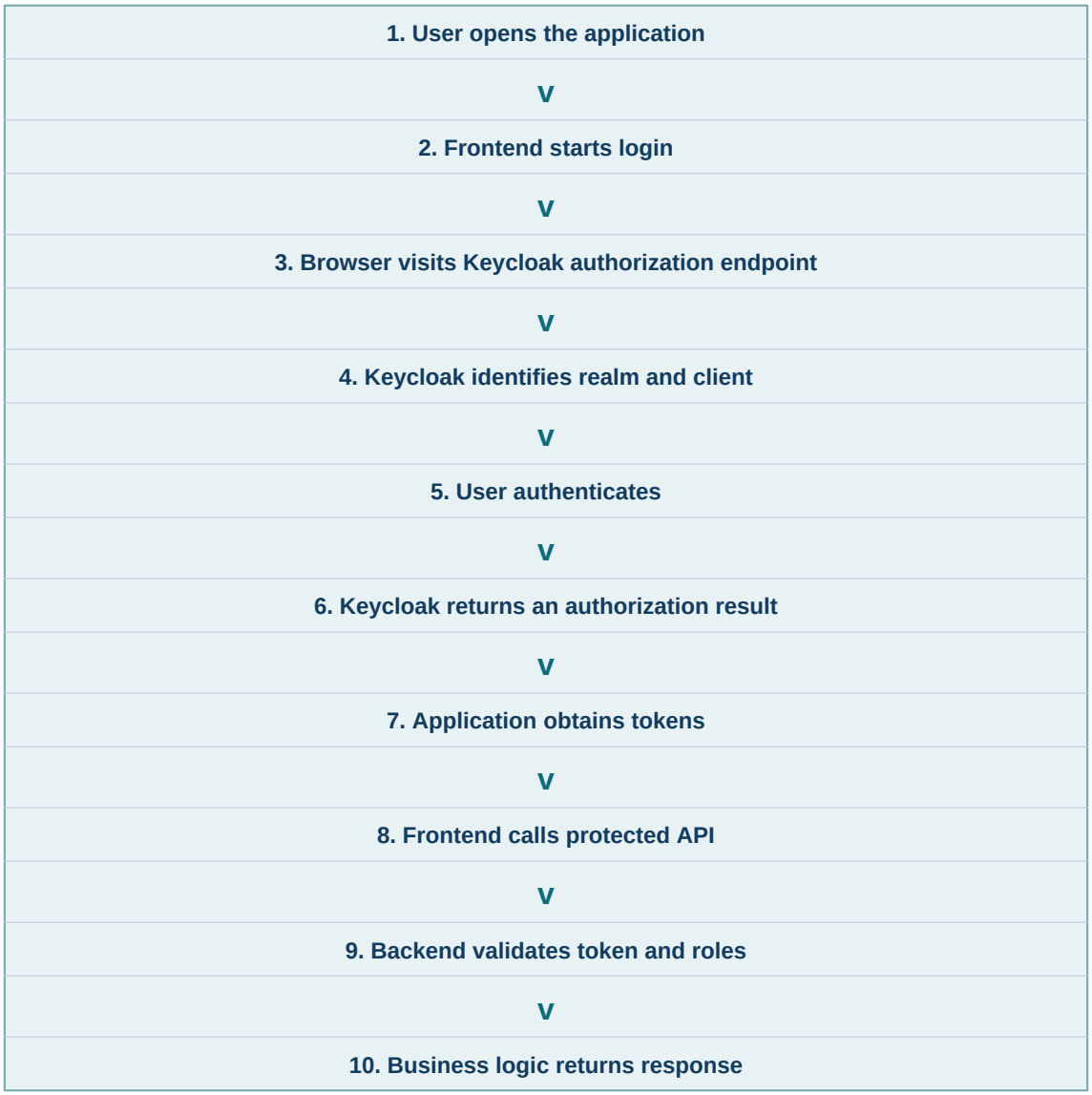
This companion PDF explains the workflows required by the pasted prompt. It is not a claim that every step is implemented in the current repository. To convert the reference workflows into project documentation, verify each step against actual files, configuration, requests, and tests.

User
V
Frontend application
V
Keycloak realm and client
V
Authentication and authorization
V
Tokens
V
Protected backend API
V
Response shown to user

The central mental model: identity is verified by Keycloak; the API trusts only a validated access token.

1. The complete system workflow

A Keycloak application is a chain of responsibilities. The frontend starts the user journey, Keycloak proves identity and issues tokens, and the backend independently checks the access token before serving protected data. The frontend is not the authority for API access.



Each arrow represents a request, redirect, local state change, or server-side validation step.

How to read this workflow

The browser may move between the application and Keycloak several times. A redirect is not the same as an API call: it changes where the browser is located. The authorization code is also not the access token. It is a short-lived result that the correct client exchanges at the token endpoint.

The backend repeats security checks even if the frontend already displays the user as logged in. This protects the API when someone calls it directly, modifies browser state, or replays an old request.

2. Login and Authorization Code Flow

The authorization code flow separates the browser-facing login step from the server-facing token exchange. This reduces the chance that long-lived credentials or tokens are exposed in the browser URL.

User clicks Login
V
Frontend builds authorization request
V
Browser redirects to Keycloak
V
Keycloak shows login page
V
User submits credentials
V
Keycloak validates user
V
Keycloak redirects with temporary code
V
Client exchanges code at token endpoint
V
Keycloak returns access, ID, and possibly refresh tokens

A code is temporary and single-purpose. It must be exchanged, not sent to a protected API.

Step-by-step

1. The frontend chooses the realm and client through its configuration. 2. It sends the browser to the authorization endpoint with values such as client_id, redirect_uri, response_type=code, and scope. 3. Keycloak validates that the client and redirect URI are allowed. 4. Keycloak authenticates the user inside the selected realm. 5. Keycloak sends the browser back to the registered redirect URI with a temporary code. 6. The client sends the code, redirect URI, and client credentials or verifier to the token endpoint. 7. Keycloak validates the exchange and issues tokens.

Failure points include an unknown realm, wrong client ID, redirect URI mismatch, disabled user, invalid credentials, expired code, or a client that is not allowed to use the selected flow. The documentation must name the exact file and function that starts the request and handles the callback.

3. PKCE: protecting the code exchange

PKCE adds a proof that the same application instance that started login is the one exchanging the authorization code. It is especially important for public clients that cannot keep a client secret safely.

The verifier stays private to the application instance; the challenge travels through the authorization request.

Why it matters

Without PKCE, an attacker who obtains an authorization code may try to exchange it before the legitimate application. With PKCE, the attacker also needs the verifier. The verifier should be high-entropy, single-use, and associated with the login transaction.

If the repository does not implement PKCE, say so explicitly. Do not describe PKCE as implemented merely because the Keycloak server supports it. Record the client type, library options, generated parameters, and callback behavior as evidence.

4. Token lifecycle

Tokens are different documents with different audiences. The access token is for the resource server, the ID token is for the client application, and the refresh token is used to obtain new access tokens when allowed.

Access token Sent in the Authorization header to a protected API. The API validates its signature and claims before using it.	**ID token** Describes the authenticated user to the client. It is not a general-purpose API permission token.
Refresh token Used at the token endpoint to request a new access token. It needs stronger protection because it can extend a session.	**Authorization code** A temporary exchange artifact. It should not be stored as the user's API credential.

Lifecycle workflow

Login succeeds
V
Code exchanged
V
Access token used for API
V
Access token expires or is near expiry
V
Refresh request sent to Keycloak
V
New access token returned
V
If refresh fails, user signs in again

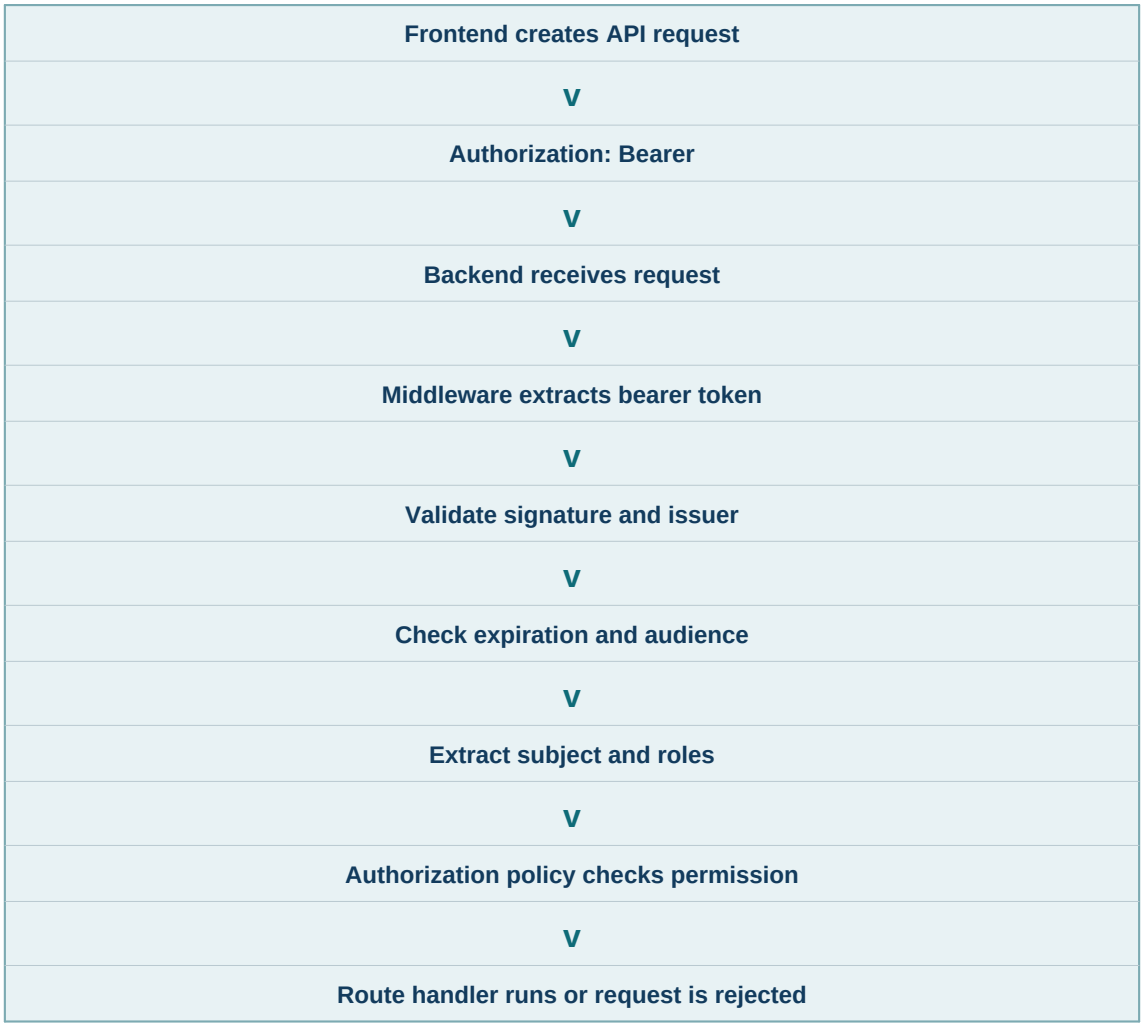
The exact storage method, rotation policy, and expiry values must be verified from project code and Keycloak configuration.

JWT validation is not decryption

A signed JWT can be decoded by anyone who has it. The signature proves that an issuer with the corresponding signing key created it and that the contents were not changed. The API must also check issuer, audience when applicable, expiration, and any required scopes or roles.

5. Protected API and token validation

A protected API request should fail closed. A missing, malformed, expired, incorrectly issued, or insufficiently authorized token must not reach protected business logic.



Authentication answers: who are you? Authorization answers: may you do this action?

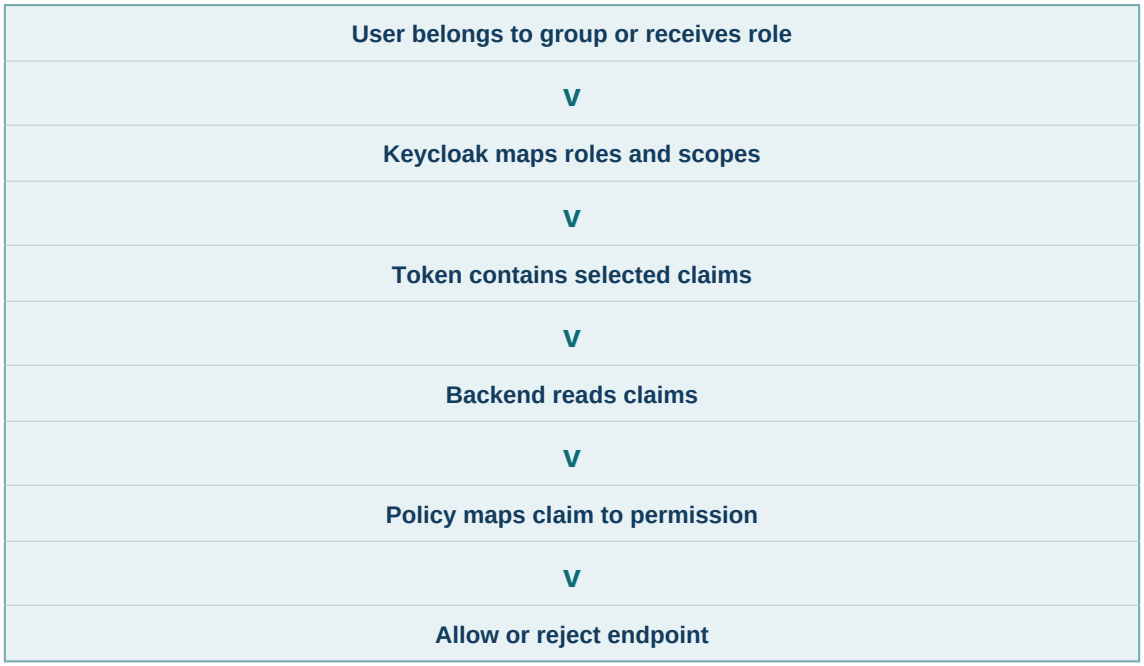
Typical decision outcomes

401 Unauthorized means the request lacks a valid authentication credential. Examples include no header, malformed token, invalid signature, wrong issuer, or expired token. 403 Forbidden means authentication succeeded but the identity lacks the required permission or role. The exact status and response body must be verified in the backend implementation.

The safest middleware order is: parse the request, extract the token, validate cryptography and claims, attach a minimal identity object to the request, check authorization, then call business logic. Logging should avoid printing full tokens.

6. Roles, scopes, and authorization

Keycloak can carry authorization information through realm roles, client roles, groups, scopes, and protocol mappers. The application still needs a clear policy describing which claim grants which action.



A role only becomes useful when the backend checks it consistently at a protected decision point.

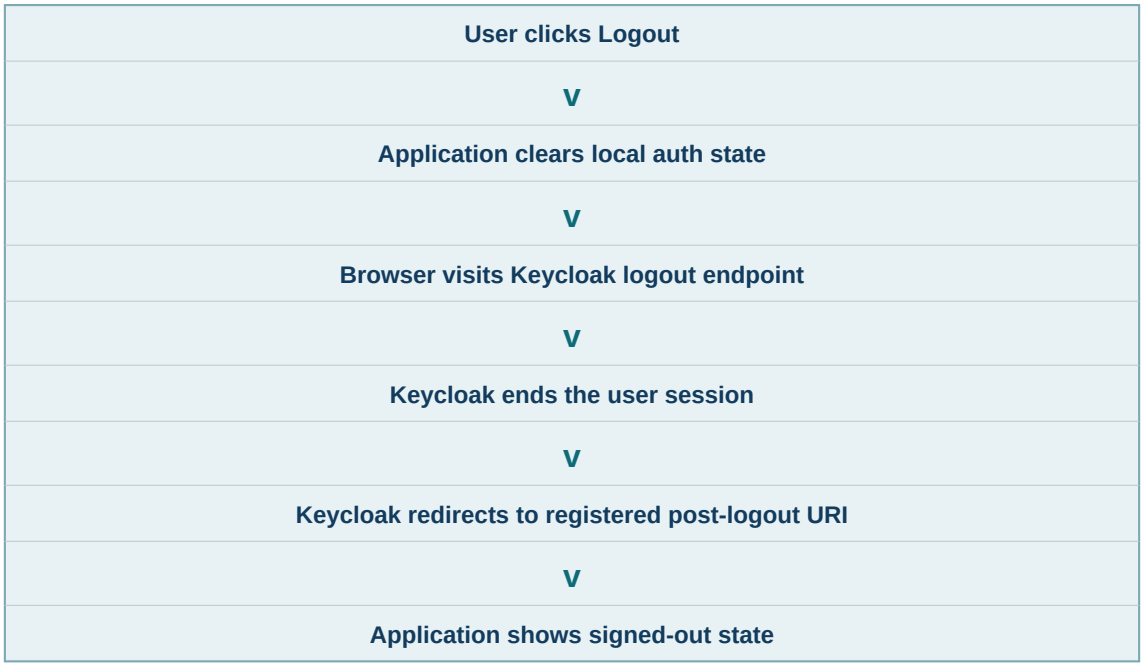
Example reasoning

Suppose an endpoint requires an admin client role. The documentation should show where that role is created, how it is assigned, which token claim carries it, how the backend extracts it, and which middleware checks it. If a role exists in the realm but no code checks it, it is configuration, not effective authorization.

Scopes are not automatically the same as roles. A scope often describes requested access or claims, while a role usually expresses an authorization position. Protocol mappers decide which data appears in tokens. These relationships must be confirmed from the realm export and application code.

7. Logout and refresh workflows

Logout has both a Keycloak session side and an application side. Ending one without the other can leave confusing behavior, such as a local screen that appears logged out while a browser session can silently log in again.



The redirect URI must be registered and should not be accepted from arbitrary user input.

Refresh workflow

When the access token expires, the application may send the refresh token to the token endpoint. Keycloak checks the session, client, token validity, and rotation or reuse policy. A successful response replaces the usable access token. A failed response should clear stale local state and require a fresh login.

Document whether the project stores tokens in memory, browser storage, cookies, or a library-managed session. The choice affects exposure to XSS, CSRF, tab persistence, and logout behavior.

8. Docker, startup, and database workflow

Local Keycloak projects often use containers to make the server, database, and configuration repeatable. Startup order is not the same as readiness: a container can be running before Keycloak can accept requests.

Docker Compose loads services and environment
V
Database starts
V
Keycloak starts
V
Keycloak reads database and realm import
V
Health/readiness becomes available
V
Application connects to Keycloak
V
User can log in

The exact import behavior and environment variable names must be verified from compose files and startup commands.

Identity data versus business data

Keycloak's database stores identity-management data such as users, clients, sessions, credentials metadata, roles, and realm configuration. The application database stores business records. They may be related by a stable subject identifier, but the application should not copy or manage passwords.

Document startup failures separately from authentication failures. A database connection error may prevent Keycloak from starting at all; a bad redirect URI may allow Keycloak to run but break login.

9. Debugging workflow

Debugging is fastest when each layer is checked in order. Begin with the browser-visible URL and only move deeper after the previous layer is correct.

Browser URL and network tab
V
Realm and client
V
Redirect URI and web origin
V
User and credentials
V
Authorization code callback
V
Token response
V
Authorization header
V
Backend issuer/signature/expiry
V
Roles and policy
V
Business response

Use the first failing layer as the starting point; later symptoms may only be consequences.

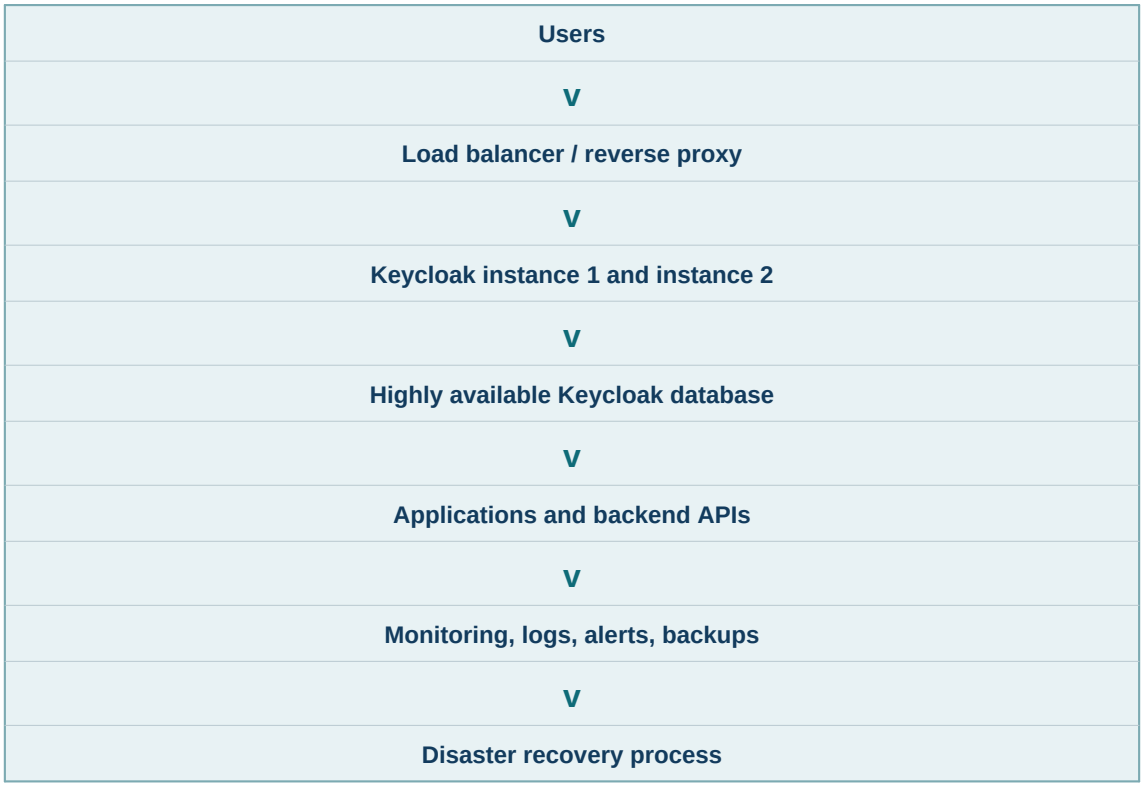
Common diagnosis

A redirect URI error points to client configuration. A login loop often points to callback handling or lost state. A 401 points to the token or validation configuration. A 403 points to role mapping or policy. A CORS error points to browser origin rules and does not necessarily mean the token is invalid.

Capture request method, URL without secrets, status code, safe response details, issuer, audience, expiry, and relevant source location. Never paste full access or refresh tokens into logs, tickets, or documentation.

10. Production architecture workflow

A simple local setup can evolve into a production identity platform by adding redundancy, durable storage, secure networking, observability, and operational controls.



Production architecture adds operational reliability around the same authentication and authorization concepts.

Scaling decisions

Multiple Keycloak instances require compatible configuration and a database strategy that supports concurrent operation. Health checks should distinguish process health from readiness. Backups must be tested by restoring them. Monitoring should cover login failures, token failures, latency, error rates, database health, CPU, memory, and active sessions.

Security improvements include HTTPS, strict redirect URIs, protected secrets, short access-token lifetimes, safe refresh-token handling, brute-force protection, MFA where appropriate, rate limiting, least privilege, and centralized audit logs.

11. The exact Login answer

When asked, 'What exactly happens when I click Login?', use this sequence and replace the generic names with verified filenames and functions from the repository.

1. User clicks Login
V
2. Frontend starts authentication
V
3. Browser redirects to Keycloak
V
4. Keycloak selects realm and client
V
5. User enters credentials
V
6. Keycloak authenticates user
V
7. Keycloak returns authorization code
V
8. Application exchanges code for tokens
V
9. Frontend stores or manages auth state
V
10. Frontend sends access token to API
V
11. Backend validates token
V
12. Backend checks roles
V
13. Business logic executes
V
14. Response returns to frontend

For every step, document what, why, where, which file/function, request, response, token, and failure behavior.

Beginner summary

Keycloak is the identity authority. The application asks Keycloak to authenticate the user. Keycloak returns proof in the form of tokens. The backend does not trust the browser's claim that the user is logged in; it trusts only a valid access token and an authorization policy that allows the requested operation.

Repository-specific completion means replacing every generic actor with evidence: actual realm name, client ID, redirect URI, library, callback handler, middleware, endpoint, role claim, and database configuration. If evidence is absent, write that the feature is not currently implemented.

12. Documentation checklist

Use this final checklist while converting the diagrams into repository-specific documentation.

Workflow area	Evidence required
Login	Frontend trigger, authorization URL, callback, code exchange
Keycloak	Version, realm, client, redirect URIs, user and role configuration
Tokens	Token library, claims used, storage, expiry and refresh behavior
API	Routes, middleware order, status codes, authorization checks
Security	HTTPS, secrets, CORS, CSRF, PKCE, MFA, rate limits
Operations	Docker, database, readiness, logs, backups, monitoring
Accuracy	Implemented versus recommended; no secrets or real tokens

Final principle

Explain the system as a chain of evidence, not as a list of Keycloak features. Every diagram arrow should correspond to a real request, redirect, configuration relationship, validation step, or documented recommendation.